Introduction to Python

Python is a high-level, interpreted programming language created by Guido van Rossum and first released in 1991.

Definition: Python is an interpreted, object-oriented, high-level programming language with dynamic semantics. It emphasizes code readability and simplicity compared to other programming languages.

- Python Named after the British comedy series "Monty Python's Flying Circus," Python has evolved into one of the most popular and versatile programming languages in the world.
- it is widely used in web development, data science, artificial intelligence, automation, and scientific computing.
- It allowing programmers to express concepts in fewer lines of code compared to languages like C++ or Java.
- Python supports multiple programming paradigms, including procedural, object-oriented, and functional programming styles.

Key Features

- **Interpreted Language**: Python code is executed line by line by the Python interpreter, eliminating the need for compilation. This makes development faster and debugging easier, as you can test code immediately without a separate compilation step.
- **Dynamic Typing**: Variables in Python don't require explicit type declarations. The interpreter automatically determines the type of a variable at runtime based on the value assigned to it, making code more flexible and concise.
- **Extensive Standard Library**: Python comes with a comprehensive standard library that provides modules and functions for tasks ranging from file I/O and system calls to web services and data manipulation.
- **Cross-Platform Compatibility**: Python runs on multiple operating systems including Windows, macOS, Linux, and Unix, allowing developers to write code once and run it anywhere without modification.
- **Object-Oriented Programming**: Python supports object-oriented programming principles such as **Abstraction** ,**encapsulation**, **inheritance**, and **polymorphism**, enabling developers to create reusable and modular code.
- **Easy to Learn and Read**: Python's syntax is clean and resembles natural English, with significant whitespace indentation that enforces readable code structure. This makes it an excellent choice for beginners while remaining powerful for experts.
- **Large Ecosystem**: Python has a vast collection of third-party packages and frameworks available through the **Python Package Index (PyPI)**, covering virtually every domain from web frameworks like **Django** and **Flask** to scientific libraries like **NumPy** and **pandas**.

Dhruv Patel16120

Characteristics

- **Indentation-Based Syntax**: Unlike languages that use braces or keywords to define code blocks, Python uses whitespace indentation. This enforces clean, readable code and eliminates debates about formatting styles.
- **Multiple Programming Paradigms**: Python supports:

    1. procedural programming (step-by-step instructions),
    2. object-oriented programming (organizing code around objects),
    3. functional programming (treating computation as mathematical functions).

- **Memory Management**: Python features automatic memory management with a **built-in garbage collector** that handles memory allocation and deallocation, freeing developers from manual memory management concerns.
- **Exception Handling**: Python provides robust exception handling mechanisms through try-except blocks, allowing developers to write resilient code that gracefully handles errors.
- **Interactive Mode**: Python offers an interactive interpreter **(REPL - Read-Eval-Print Loop)** that allows developers to execute code snippets immediately and experiment with ideas in real-time.
- **Extensibility**: Python can be extended with modules written in C or C++ for performance-critical operations, and it can also be embedded into other applications to provide scripting capabilities.

Interesting Facts

- Python consistently ranks among the top three programming languages in popularity indices such as TIOBE and Stack Overflow surveys. Its growth has been particularly explosive in data science and machine learning fields.
- Major companies including Google, Netflix, Instagram, Spotify, and Dropbox use Python extensively in their production systems. Google has been a particularly strong supporter, with Python being one of its three official programming languages alongside C++ and Java.
- Python 2 and Python 3 are two major versions with significant differences. Python 2 reached end-of-life on January 1, 2020, and the community has since fully transitioned to Python 3, which includes improvements in syntax, Unicode support, and performance.

Applications of Python Across Different Domains

Python's versatility makes it the go-to language across numerous industries. Here's a concise overview of the essential tools and frameworks for each domain.

Web Development

- **Django** - Full-featured framework with ORM, admin panel, and built-in security. Used by Instagram and Pinterest for large-scale applications.
- **Flask** - Lightweight micro-framework offering flexibility for APIs and small to medium apps. Powers services at Netflix and Reddit.

DhruvPatel1612

- **FastAPI** - Modern, high-performance framework for building APIs with automatic documentation and data validation.

Machine Learning

- **Scikit-learn** - Comprehensive library for classical ML algorithms including classification, regression, and clustering.
- **XGBoost/LightGBM** - High-performance gradient boosting libraries, dominant in data competitions and production systems.

Deep Learning & AI

- **TensorFlow** (Google) - End-to-end platform for neural networks with deployment across mobile, web, and embedded systems.
- **PyTorch** (Meta) - Research-friendly framework with dynamic computation graphs, preferred in academia and increasingly in industry.
- **Keras** - High-level API integrated with TensorFlow for rapid model prototyping.
- **Hugging Face Transformers** - State-of-the-art pre-trained models for NLP tasks like text generation and classification.

Data Science & Analytics

- **Pandas** - Essential for data manipulation with DataFrame structures for cleaning, transforming, and analyzing data.
- **NumPy** - Foundation for numerical computing with efficient array operations.
- **Matplotlib/Seaborn** - Visualization libraries for static plots, with Seaborn offering statistical graphics.
- **Plotly** - Interactive visualizations for dashboards and web applications.
- **Jupyter Notebooks** - Interactive environment combining code, visualizations, and documentation.

Scientific Computing

- **SciPy** - Advanced mathematics, optimization, signal processing, and statistical functions built on NumPy.
- **SymPy** - Symbolic mathematics for algebraic manipulations.
- **Biopython** - Computational biology tools for sequence analysis and bioinformatics.

Automation & Scripting

- **Selenium** - Browser automation for web testing and scraping.
- **Beautiful Soup/Scrapy** - HTML parsing and web crawling frameworks for data extraction.
- **PyAutoGUI** - GUI automation controlling mouse and keyboard actions.

Cybersecurity

- **Scapy** - Packet manipulation and network security testing.
- **Nmap (python-nmap)** - Network scanning and security auditing automation.
- **Impacket** - Network protocol manipulation for penetration testing.

DhruvPatel16120

IoT & Embedded Systems

- **MicroPython/CircuitPython** - Python implementations for microcontrollers like ESP32 and Raspberry Pi Pico.
- **Raspberry Pi** - Single-board computer using Python for electronics projects and robotics.
- **Paho-MQTT** - IoT messaging protocol implementation.

Finance & Trading

- **QuantLib** - Quantitative finance modeling and risk management.
- **Backtrader** - Algorithmic trading strategy backtesting framework.
- **yfinance** - Financial market data access and analysis.

Desktop GUI Applications

- **PyQt/PySide** - Professional cross-platform applications using Qt framework. Used by Dropbox and Calibre.
- **Tkinter** - Built-in GUI library for simple applications.
- **Kivy** - Multi-touch apps working across desktop and mobile platforms.

Cloud Computing & DevOps

- **Boto3** - AWS SDK for managing cloud services like S3, EC2, and Lambda.
- **Google Cloud/Azure SDKs** - Official libraries for respective cloud platforms.
- **Docker SDK/Kubernetes Client** - Container and orchestration management.

Natural Language Processing

- **NLTK** - Comprehensive NLP toolkit for tokenization, parsing, and classification.
- **spaCy** - Industrial-strength NLP with fast processing and pre-trained models.
- **Gensim** - Topic modeling and document similarity using Word2Vec and LDA.

Computer Vision

- **OpenCV** - Industry-standard library for image processing, object detection, and video analysis.
- **Pillow** - Image file manipulation and basic processing.
- **MediaPipe** - Google's framework for real-time hand tracking, face detection, and pose estimation.

Game Development

- **Pygame** - 2D game development with graphics and sound modules.
- **Panda3D** - 3D game engine originally developed by Disney.

Variables and Data Types in Python

Variables: The Basics

Definition: Variables are containers for storing data. Python uses **dynamic typing**—no need to declare types; the interpreter figures it out automatically.

DhruvPatel16120

```
# Simple assignment
name = "Alice"
age = 25

# Multiple assignment
x, y, z = 10, 20, 30

# Same value to multiple variables
a = b = c = 0
```

Naming Rules

- **Must Follow:**
  - Start with letter (a-z, A-Z) or underscore (_)
  - Only letters, numbers, and underscores
  - Case-sensitive: `age`, `Age`, and `AGE` are different
  - No Python keywords (like `if`, `for`, `class`)
- **Best Practices:**
  - Use `snake_case`: `user_name`, `total_price`
  - Be descriptive: `student_count` not `sc`
  - Constants in UPPERCASE: `MAX_SIZE`, `PI`

- ### Numeric Types

Type	Description	Example	Notes
int	Whole numbers	`42, -17, 0`	Unlimited precision
float	Decimal numbers	`3.14, -0.5`	~15 -17 digit precision
complex	Complex numbers	`3+4j, 2j`	Real + imaginary parts

- ### Text Type

Type	Description	Example	Key Feature
str	Text/characters	`"Hello", 'Python'`	Immutable, Unicode support

- ### Boolean Type

Type	Values	Usage
bool	`True, False`	Conditions, comparisons, logical operations

- ### Special Type

Type	Value	Purpose
NoneType	`None`	Represents absence of value

Type Checking :

➢ **type():**The type() function is used to **identify the data type of a variable or value**. It returns the **class type** of the object.

Dhruvpatel16120

```
# Displaying the data types
print("Data type of a (10) is:", type(a))
```

- **isinstance(variable name,Data Type):** The isinstance() function is used to check whether a variable belongs to a specific data type. It returns **True** if the variable matches the given type, otherwise it returns **False**.

```
# check instance type of variables
print("instance type of b (3.14) is:", isinstance(b, float))
```

- **id(obj)**: The id() function in Python returns the unique identity (memory address) of an object.This identity is guaranteed to be unique and constant for the object during its lifetime.

```
# print memory address of variables
print(id(a))
```

2.1 Collection Data Types in Python

Collection data types are used to store **multiple values in a single variable**. They help organize data efficiently and allow different operations such as adding, removing, or searching elements. Python provides four main built-in collection data types: **List, Tuple, Set, and Dictionary**.

List

A list is an **ordered and mutable** collection of elements. This means the items have a fixed order, and you can change, add, or remove elements after the list is created. Lists also allow duplicate values.

Example:

```
numbers = [1, 2, 3, 4]
names = ["Alice", "Bob", "Charlie"]
```

Lists are commonly used when data needs to be modified frequently.

Tuple

A tuple is an **ordered but immutable** collection. Once a tuple is created, its elements cannot be changed. Like lists, tuples allow duplicate values.

Example:

```
coordinates = (10, 20)
days = ("Mon", "Tue", "Wed")
```

Tuples are useful when data should remain constant, such as fixed configuration values.

DhruvPatel16120

Set

A set is an **unordered and mutable** collection of unique elements. Sets do not allow duplicate values, and the order of elements is not guaranteed.

Example:

```
unique_numbers = {1, 2, 3, 3}
# Output will be: {1, 2, 3}
```

Sets are mainly used when you want to store unique values or perform mathematical set operations.

Dictionary

A dictionary is a collection that stores data in **key–value pairs**. Each key must be unique and immutable, while values can be changed.

Example:

```
student = {
    "name": "Alice",
    "age": 20,
    "grade": "A"
}
```

Dictionaries are widely used to represent structured data like records, JSON objects, or database rows.

`range`

The `range` data type is used to generate a sequence of numbers. It is commonly used in loops. The numbers are generated **on demand**, which makes `range` memory-efficient.

Example:

```
range_example = range(1, 10)
```

Explanation:
This creates a sequence starting from `1` up to `9`. The ending value `10` is not included. The `range` object itself does not store all values in memory; instead, it generates them when needed.

DhruvPatel16120

bytes

The `bytes` data type represents a sequence of **immutable binary data**. It is mainly used for handling raw data such as files, images, or network communication.

Example:

```
byte_example = b'Hello'
```

Explanation:
This creates a bytes object containing the binary representation of the word `"Hello"`. Once created, the data cannot be changed.

---

bytearray

The `bytearray` data type is similar to `bytes`, but it is **mutable**, meaning its values can be modified after creation.

Example:

```
bytearray_example = bytearray(5)
```

Explanation:
This creates a bytearray of size `5` with all elements initialized to `0`. It is useful when working with binary data that needs to be modified.

---

memoryview

The `memoryview` data type allows direct access to the memory of another binary object without copying it. It improves performance when working with large data.

Example:

```
memoryview_example = memoryview(bytearray(5))
```

Explanation:
This creates a memory view of a bytearray. Any change made through the memoryview will affect the original bytearray.

---

frozenset

A `frozenset` is an **immutable version of a set**. It stores unique elements but does not allow modification after creation.

DhruvPatel16120

Example:

```
frozenset_example = frozenset([1, 2, 3, 4, 5])
```

Explanation:
This creates a frozenset containing unique numbers. Since it is immutable, elements cannot be added or removed. Frozensets are often used as dictionary keys or elements of another set.

Summary

- **List**: Ordered, changeable, allows duplicates
- **Tuple**: Ordered, unchangeable, allows duplicates
- **Set**: Unordered, changeable, no duplicates
- **Dictionary**: Key–value pairs, keys are unique
- **`range`**: Generates a sequence of numbers efficiently
- **`bytes`**: Immutable sequence of binary data
- **`bytearray`**: Mutable sequence of binary data
- **`memoryview`**: Access and modify binary data without copying
- **`frozenset`**: Immutable collection of unique elements

Python Keywords: Python keywords are reserved words that have special meaning in the Python language. They cannot be used as variable names, function names, or identifiers because Python uses them to define syntax and structure.

Keyword	Category	Basic Definition	Simple Example
`False`	Boolean	Represents a false value	`is_active = False`
`True`	Boolean	Represents a true value	`is_valid = True`
`None`	Special	Represents no value	`result = None`
`and`	Logical	Returns True if both conditions are true	`x > 5 and x < 10`
`or`	Logical	Returns True if at least one condition is true	`x > 5 or x < 3`
`not`	Logical	Reverses a condition	`not is_logged_in`
`if`	Conditional	Executes code if condition is true	`if x > 0:`
`elif`	Conditional	Checks another condition	`elif x == 0:`
`else`	Conditional	Executes if all conditions fail	`else:`
`for`	Loop	Iterates over a sequence	`for i in range(5):`
`while`	Loop	Runs while condition is true	`while x < 5:`
`break`	Loop Control	Stops the loop	`break`
`continue`	Loop Control	Skips current iteration	`continue`
`pass`	Placeholder	Does nothing (empty block)	`pass`
`def`	Function	Defines a function	`def add(a, b):`
`return`	Function	Sends result back from function	`return a + b`
`lambda`	Function	Creates anonymous function	`lambda x: x * 2`
`class`	OOP	Defines a class	`class User:`
`self`	OOP	Refers to current object	`self.name = name`

mwatech6120

`import`	Module	Imports a module	`import math`
`from`	Module	Imports specific items	`from math import sqrt`
`as`	Module	Gives alias to module	`import numpy as np`
`try`	Exception	Tests code for errors	`try:`
`except`	Exception	Handles error	`except ValueError:`
`finally`	Exception	Executes always	`finally:`
`raise`	Exception	Raises an exception	`raise ValueError()`
`with`	Context	Manages resources	`with open("file.txt") as f:`
`in`	Membership operator	Checks presence in sequence	`x in list`
`is`	Identity operator	Checks same object	`x is None`
`global`	Scope	Declares global variable	`global count`
`nonlocal`	Scope	Refers to outer function variable	`nonlocal x`
`assert`	Debugging	Tests condition	`assert x > 0`
`yield`	Generator	Returns value & pauses function	`yield i`
`async`	Async	Declares async function	`async def fetch():`
`await`	Async	Waits for async result	`await fetch()`

Scope of Variables in Python

Scope refers to the region where a variable can be accessed in a program.

Python follows the **LEGB rule** to resolve variables.

□ Types of Scope (LEGB Rule)

Scope	Description
Local	Inside current function
Enclosing	Inside outer (nested) function
Global	Defined outside all functions
Built-in	Predefined functions (len, print)

🔑 **Keywords**

- global → modify global variable
- nonlocal → modify enclosing variable

✅ **Single Example (Covers All Scopes)**

```
x = "global"   # Global scope

def outer():
    x = "enclosing"   # Enclosing scope

    def inner():
```

Dhruv Patel16

```
        x = "local"   # Local scope
        print("Local:", x)

    inner()
    print("Enclosing:", x)

outer()
print("Global:", x)

print("Built-in:", len("Python"))  # Built-in scope
```

Output

Local: local

Enclosing: enclosing

Global: global

Built-in: 6

Key Points

- Python searches variables in **LEGB order**
- Local → highest priority
- Global variables require global keyword to modify
- Enclosing variables use nonlocal
- Built-in names are always available

Identifiers in Python

Definition: Identifiers are the **names used to identify variables, functions, classes, objects, and other user-defined items** in a Python program.

Rules for Identifiers

- Must start with a **letter (A–Z or a–z)** or an **underscore (_)**
- Can contain **letters, digits (0–9), and underscores**
- **Cannot start with a digit**
- **Case-sensitive** (`count`, `Count`, and `COUNT` are different)
- Must **not be a Python keyword** (such as `if`, `for`, `class`, `while`)

Examples

Valid identifiers: `Name` ,`student_count`,`_total`,`userAge`

Invalid identifiers: `2value`,`total-price`, class

DhruvPatel16120

Best Practices

- Use **snake_case** for variable and function names
- Use **descriptive names** for better readability
- Use **UPPERCASE** for constants

Escape Sequences in Python

Definition: Escape sequences are **special character combinations** used inside strings to represent characters that are difficult or impossible to type directly.

They start with a **backslash (\)** followed by a character.

Escape Sequence Table

Escape Sequence	Description	Example	Output
`\n`	New line	`"Hello\nWorld"`	Hello World
`\t`	Horizontal tab	`"Hello\tWorld"`	Hello World
`\'`	Single quote	`'It\'s Python'`	It's Python
`\"`	Double quote	`"He said \"Hi\""`	He said "Hi"
`\\`	Backslash	`"C:\\Path"`	C:\Path
`\r`	Carriage return	`"Hello\rWorld"`	World
`\b`	Backspace	`"Hello\b"`	Hell
`\uXXXX`	Unicode character	`"\u03A9"`	Ω
`\xhh`	Hex value	`"\x48"`	H

Operators in Python

Definition: Operators are **symbols used to perform operations on variables and values**. They allow Python programs to perform calculations, make decisions, and manipulate data.

1. Arithmetic Operators :*Arithmetic operators are used to perform mathematical calculations.*

Operator	Description	Example	Result
+	Addition	`10 + 3`	`13`
-	Subtraction	`10 - 3`	`7`
*****	Multiplication	`10 * 3`	`30`
/	Division	`10 / 3`	`3.333`
//	Floor division	`10 // 3`	`3`
%	Modulus	`10 % 3`	`1`
******	Exponent	`2 ** 3`	`8`

2. Assignment Operators : Assignment operators are used to **assign and update values** in variables.

Operator	Description	Example
=	Assign value	`x = 5`
+=	Add and assign	`x += 3`

DhruvPatel120

-=	Subtract and assign	`x -= 2`
***=**	Multiply and assign	`x *= 4`
/=	Divide and assign	`x /= 2`
//=	Floor divide and assign	`x //= 2`
%=	Modulus and assign	`x %= 2`
****=**	Exponent and assign	`x **= 2`

3. Comparison (Relational) Operators: Comparison operators are used to **compare two values** and return `True` or `False`.

Operator	Description	Example
==	Equal to	`x == y`
!=	Not equal to	`x != y`
>	Greater than	`x > y`
<	Less than	`x < y`
>=	Greater than or equal to	`x >= y`
<=	Less than or equal to	`x <= y`

4. Logical Operators: Logical operators are used to **combine conditions**.

Operator	Description	Example
And	True if both conditions are true	`x > 0 and x < 10`
Or	True if at least one condition is true	`x > 0 or x < 0`
Not	Reverses the result	`not(x > 0)`

5. Bitwise Operators

Bitwise operators perform operations on **individual bits (0s and 1s)** of integer numbers. Python first converts numbers into **binary form**, then applies the operation **bit by bit**.

Bitwise operators work **only with integers**, not with floating-point numbers.

Binary Representation (Basic Idea)

Every integer is stored in binary.

Example:

- `5 → 101`
- `3 → 011`

Types of Bitwise Operators in Python

1. Bitwise AND (&) : Compares each bit of two numbers. Result bit is 1 **only if both bits are 1**.

Example:
```
5 & 3
```

Binary:

DhruvPatel16it

```
5 → 101
3 → 011
---------
    001
```

Result: 1

2. Bitwise OR (|) :Result bit is `1` if **any one of the bits is 1**.

Example:
`5 | 3`

Binary:
```
5 → 101
3 → 011
---------
    111
```

Result: 7

3. Bitwise XOR (^)

Result bit is `1` if **bits are different**.

Example:
`5 ^ 3`

Binary:
```
5 → 101
3 → 011
---------
    110
```

Result: 6

---

4. Bitwise NOT (~)

Inverts all bits (0 becomes 1, 1 becomes 0).

Example:
`~5`

Result: `-6`

Explanation: Python uses **two's complement** to represent negative numbers, which is why the result is `-6`.

---

5. Left Shift (<<)

Shifts bits to the **left** by a given number of positions.
Each left shift **multiplies the number by 2**.

Example:
`5 << 1`

Binary:
`101 → 1010`

DhruvPatel16120

Result: 10

---

6. Right Shift (>>)

Shifts bits to the **right**.
Each right shift **divides the number by 2** (floor division).

Example:
`5 >> 1`

Binary:
`101 → 10`

Result: 2

Bitwise operators perform operations at the **binary level**.

Operator	Description	Example
&	AND	`5 & 3`
`	`	OR
^	XOR	`5 ^ 3`
~	NOT	`~5`
<<	Left shift	`5 << 1`
>>	Right shift	`5 >> 1`

---

6. Membership Operators

Membership operators are used to **check whether a value exists** in a sequence.

Operator	Description	Example
`in`	Returns True if value exists	`a in list`
`not in`	Returns True if value does not exist	`a not in list`

7. Identity Operators

Identity operators are used to **compare memory locations** of objects.

It compare memory location not values of variables, so it is different from == .

Operator	Description	Example
`is`	True if same object	`a is b`
`is not`	True if different objects	`a is not b`

Type Casting in Python

Definition: Type casting is the process of **converting one data type into another**. In Python, type casting is commonly used to convert data entered by the user or to perform operations between different data types.

DhruvPatel1120

Python supports **explicit type casting**, where the programmer manually converts one data type into another using built-in functions.

Types of Type Casting in Python

Python has **two types of type casting**:

1. Implicit Type Casting

Implicit type casting is done **automatically by Python**. Python converts a smaller data type into a larger one to avoid data loss.

Example:

```
a = 10        # int
b = 2.5       # float
c = a + b     # int converted to float

print(c)      # 12.5
print(type(c))  # <class 'float'>
```

Explanation: Here, Python automatically converts `int` to `float`.

---

2. Explicit Type Casting

Explicit type casting is done **manually by the programmer** using built-in functions.

Common type casting functions:

- `int()`
- `float()`
- `str()`
- `bool()`

`int()` Function

Definition: The `int()` function converts a value into an **integer**.

Example:

```
x = int(3.14)
y = int("10")

print(x)  # 3
print(y)  # 10
```

DhruvPatel16120

Important Points:

- Decimal part is **removed**, not rounded
- Strings must contain valid numbers

Invalid example: `int("10.5")  # Error`

`float()` Function

Definition: The `float()` function converts a value into a **floating-point number**.

Example:

```
x = float(10)
y = float("3.14")

print(x)  # 10.0
print(y)  # 3.14
```

`str()` Function

Definition: The `str()` function converts a value into a **string**.

Example:

```
age = 25
text = str(age)

print(text)
print(type(text))  # <class 'str'>
```

Used mainly for:

- Displaying output
- String concatenation
- Formatting messages

Common Use Case: User Input

```
age = int(input("Enter age: "))
height = float(input("Enter height: "))
```

Explanation: The `input()` function always returns a string, so type casting is required to perform calculations.

Type Casting Summary Table

Function	Converts To	Example

DhruvPatel16120

`int()`	Integer	`int(3.5)` → `3`
`float()`	Float	`float(5)` → `5.0`
`str()`	String	`str(10)` → `"10"`
`bool()`	Boolean	`bool(1)` → `True`

Key Points

- Type casting changes the **data type of a value**
- Python supports **implicit and explicit** casting
- Explicit casting gives more **control and clarity**
- Incorrect casting causes **runtime errors**

Type Casting with Collections in Python

Definition: Type casting with collections means **converting one collection data type into another**, such as converting a list into a tuple, set, or dictionary, using Python's built-in collection functions.

Python allows conversion between compatible collection types.

Common Collection Type Casting Functions

- `list()`
- `tuple()`
- `set()`
- `dict()`

---

1. Converting to List (`list()`)

The `list()` function converts a collection or iterable into a **list**.

Example:

```
t = (1, 2, 3)
l = list(t)
```

Explanation: The tuple is converted into a list, which is mutable.

2. Converting to Tuple (`tuple()`)

The `tuple()` function converts a collection into a **tuple**.

Example:

```
l = [10, 20, 30]
t = tuple(l)
```

DhruvPatel1612O

Explanation: The list is converted into an immutable tuple.

---

3. Converting to Set (`set()`)

The `set()` function converts a collection into a **set**.

Example:

```
l = [1, 2, 2, 3]
s = set(l)
```

Explanation: Duplicate elements are automatically removed because sets store only unique values.

---

4. Converting to Dictionary (`dict()`)

The `dict()` function converts data into a **dictionary** only if it is in a valid key–value format.

Example using list of tuples:

```
data = [("a", 1), ("b", 2)]
d = dict(data)
```

Example using tuple of tuples:

```
data = (("x", 10), ("y", 20))
d = dict(data)
```

Invalid Dictionary Conversion

```
dict([1, 2, 3])  # Error
```

Explanation: Each element must be a pair (key, value).

Type Casting Between Strings and Collections

String to List

```
text = "Python"
l = list(text)
```

Output:

```
['P', 'y', 't', 'h', 'o', 'n']
```

List to String

```
chars = ['P', 'y', 't', 'h', 'o', 'n']
text = "".join(chars)
```

DhruvPatel16120

Summary Table

From	To	Function	Result
List	Tuple	`tuple()`	Immutable collection
Tuple	List	`list()`	Mutable collection
List	Set	`set()`	Removes duplicates
List of pairs	Dictionary	`dict()`	Key–value mapping
String	List	`list()`	Characters as elements

Key Points

- Collection type casting works only between **compatible types**
- Sets automatically remove duplicates
- Dictionary conversion requires **key–value pairs**
- Order may change when converting to a set

Control Flow in Python

Definition: Control flow refers to the order in which statements in a program are executed. By default, Python executes code line by line, but control flow statements allow the program to make decisions, repeat actions, or skip code based on conditions.

Types of Control Flow in Python

Python control flow is mainly divided into **three categories**:

1. **Conditional Statements** (Decision Making)
2. **Looping Statements** (Repetition)
3. **Loop Control Statements** (Flow Control inside loops)

---

1. Conditional Statements

Conditional statements allow the program to **execute different blocks of code based on conditions**.

`if` Statement

The `if` statement executes a block of code **only if the condition is True**.

Syntax:

```
if condition:
    statement
```

Example:

DhruvPatel16120

```
age = 18
if age >= 18:
    print("Eligible to vote")
```

`if-else` Statement

The `else` block executes when the `if` condition is False.

Example:

```
age = 16
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

`if-elif-else` Statement

Used to check **multiple conditions**.

Example:

```
marks = 75

if marks >= 90:
    print("Grade A")
elif marks >= 70:
    print("Grade B")
else:
    print("Grade C")
```

Nested `if` Statement:

An `if` statement inside another `if` statement.

Example:

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
```

Looping Statements in Python

What is a Loop?

A loop is a control flow statement used to **execute a block of code repeatedly** until a condition is met or a sequence ends.
Loops help reduce repetition and make programs efficient.

DhruvPatel16120

Types of Loops in Python

Python provides **two main looping statements**:

1. **`for` loop**
2. **`while` loop**

Python also supports **nested loops** and **loop control statements**.

---

1. `for` Loop

Purpose :The `for` loop is used to **iterate over a sequence** such as:

- List
- Tuple
- String
- Set
- Dictionary
- Range

Syntax

```
for variable in sequence:
    block of code
```

How `for` Loop Works (Internally)

1. Takes the first element from the sequence
2. Assigns it to the loop variable
3. Executes the loop body
4. Moves to the next element
5. Repeats until the sequence ends

Example with `range()`

```
for i in range(1, 6):
    print(i)
```

Output:

```
1 2 3 4 5
```

DhruvPatel16120

Looping Through Collections

List

```
numbers = [10, 20, 30]
for n in numbers:
    print(n)
```

String

```
for ch in "Python":
    print(ch)
```

Dictionary

```
student = {"name": "Alice", "age": 20}

for key in student:
    print(key, student[key]) # print key and value
```

`for` Loop with `else`

The `else` block executes **only if the loop finishes normally** (no `break`).

```
for i in range(5):
    print(i)
else:
    print("Loop completed")
```

Note : If `break` is used, `else` will not execute and skiped.

---

2. `while` Loop

Purpose: The `while` loop is used when the **number of iterations is unknown** and depends on a condition.

Syntax

```
while condition:
    block of code
```

How `while` Loop Works

1. Checks the condition
2. Executes loop body if condition is True
3. Updates variables
4. Repeats until condition becomes False

DhruvPatel16120

Example

```
count = 1
while count <= 5:
    print(count)
    count += 1 # necessary to terminate loop
```

Infinite Loop

If the condition never becomes False, the loop runs forever.

```
while True:
    print("Running")
```

Use `break` to stop it safely.

---

3. Loop Control Statements (Detailed)

`break`

Terminates the loop immediately.

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

`continue`

Skips the current iteration.

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

`pass`

Acts as a placeholder.

```
for i in range(5):
    if i == 3:
        pass
    print(i)
```

---

4. Nested Loops

A loop inside another loop.

```
for i in range(1, 4):
    for j in range(1, 4):
```

DhruvPatel16120

```
        print(i, j)
```

Used for:

- Matrix operations
- Pattern printing
- Combinations

---

5. Common Loop Patterns

Sum of numbers

```
total = 0
for i in range(1, 6):
    total += i
```

Searching an element

```
nums = [1, 3, 5, 7]
found = False

for n in nums:
    if n == 5:
        found = True
        break
```

---

6. `for` vs `while` (Key Difference)

Feature	for loop	while loop
Iterations	Known	Unknown
Condition	Sequence-based	Boolean-based
Common use	Iteration	Repetition

Key Points to Remember

- `for` loops are best for sequences
- `while` loops depend on conditions
- Always update variables in `while` loops
- Avoid infinite loops
- Indentation is mandatory

Functions in Python

Definition : A function is a **block of reusable code** that performs a specific task.
Functions help in **breaking a program into smaller parts**, improving **code readability, reusability, and maintainability**.

Instead of writing the same code again and again, a function can be written once and used multiple times.

Dhruvpatel16120

Types of Functions in Python

Python mainly has **two types of functions**:

1. **Built-in Functions**
2. **User-Defined Functions**

- **Built-in Functions :**

Function	Purpose	Example
`len()`	Returns length of variables	`len([1,2,3])`
`type()`	Returns data type	`type(10)`
`id()`	Returns memory address	`id(x)`
`sum()`	Returns sum of elements	`sum([1,2,3])`
`max()`	Returns largest value	`max([3,7,1])`
`min()`	Returns smallest value	`min([3,7,1])`
`abs()`	Returns absolute value	`abs(-10)`
`pow()`	Power calculation	`pow(2,3)`
`round()`	Rounds a number	`round(3.6)`
`sorted()`	Sorts elements	`sorted([3,1,2])`
`range()`	Generates number sequence	`range(1,5)`
`enumerate()`	Return Index + value pair tuple	`enumerate(['a','b'])`
`zip()`	Combines iterables	`zip(a, b)`
`all()`	True if all values are True	`all([True, True])`
`any()`	True if any value is True	`any([False, True])`
`isinstance()`	Checks object type	`isinstance(5, int)`
`dir()`	Lists object attributes	`dir(list)`
`help()`	Shows documentation	`help(len)`
`reversed()`	Reverses iterable	`reversed([1,2,3])`
`slice()`	Creates slice object	`slice(1,4)`
`map()`	Applies function to items	`map(fn, data)`
`filter()`	Filters elements	`filter(fn, data)`
`eval()`	Evaluates expression	`eval("2+3")`
`globals()`	Returns global namespace	`globals()`
`locals()`	Returns local namespace	`locals()`
`bin()`	Converts integer to binary string	`bin(10)`
`oct()`	**Converts integer to octal string**	**`oct(10)`**
`hex()`	Converts integer to hexadecimal string	`hex(10)`

2. User Define functions :

Definition: A user-defined function is a function **created by the programmer** to perform a specific task.
It allows code to be **reusable, modular, and easier to maintain**.

User-defined functions are defined using the **`def` keyword**.

Why User-Defined Functions Are Important

- Avoid code repetition
- Improve readability
- Make debugging easier
- Enable modular programming
- Essential for large projects and frameworks

Syntax of a User-Defined Function

```
def function_name(parameters):
    function_body
    return value
```

Example

```
def add(a, b):
    return a + b

result = add(5, 3)
```

Explanation:

- `def` → defines a function
- `add` → function name
- `a, b` → parameters
- `return` → sends result back

Function Execution Flow (Concept)

1. Function is **defined** (not executed)
2. Function is **called**
3. Parameters receive values
4. Function body executes
5. Return value is sent back

Types of User-Defined Functions (Based on Arguments)

1. No arguments, no return value

```
def greet():
    print("Hello")
```

---

2. Arguments, no return value

```
def greet(name):
    print("Hello", name)
```

DhruvPatel16120

3. No arguments, return value

```
def get_pi():
    return 3.14
```

4. Arguments and return value

```
def square(x):
    return x * x
```

Function Arguments (Important Concepts)

Positional Arguments

```
def add(a, b):
    return a + b
```

Default Arguments

```
def greet(name="User"):
    print(name)
```

Keyword Arguments

```
greet(name="Alice")
```

Variable-Length Arguments

```
def total(*nums):
    return sum(nums)
```

Scope of Variables in Functions

- **Local variables** → defined inside function
- **Global variables** → defined outside function

```
x = 10 #global

def show():
    x =155 # local
    print(x)
    print(global x)
```

Variable-Length Arguments in Python (*args and **kwargs)
Definition: Variable-length arguments allow a function to accept **any number of arguments**.

1⃣*args (Non-Keyword Arguments)
Definition: *args allows a function to accept **multiple positional arguments**.
it stores values as a **tuple**.
Example
def add(*args):
  return sum(args)

DhruvPatel16120

print(add(1, 2, 3, 4)) # 10

Key Points

- Accepts unlimited positional arguments
- Stored as a tuple
- Name args is convention (can use any name)

2⃣ **kwargs (Keyword Arguments)

Definition: **kwargs allows a function to accept **multiple keyword arguments**.
It stores values as a **dictionary**.

Example

```
def display(**kwargs):
    print(kwargs)

display(name="Alice", age=20)
```

Output

{'name': 'Alice', 'age': 20}

Key Points

- Accepts key-value arguments
- Stored as dictionary
- Name kwargs is convention

⚡ **Important Rules**

*args must come **before** **kwargs
Order:def func(normal, *args, **kwargs):

Lambda Functions and Recursion in Python

What is a Lambda Function?

Definition: A lambda function is an anonymous function that can have multiple arguments but only one expression**.**

It can have **any number of arguments**, but **only one expression**. Lambda functions are mainly used for **short, simple operations**.

We assume that arguments are parameters and expression are function body that perform only one task.

Syntax of Lambda Function

```
lambda arguments : expression
```

Example

```
square = lambda x: x * x
print(square(5))
```

Explanation:

DhruvPatel16120

- `lambda` → keyword
- `x` → argument
- `x * x` → expression returned automatically

Lambda vs Normal Function

Normal function:

```
def add(a, b):
    return a + b
```

Lambda function:

```
add = lambda a, b: a + b
```

Using Lambda with Built-in Functions

`map()` : Applies a function to each element.

```
numbers = [1, 2, 3, 4]
result = list(map(lambda x: x * 2, numbers))
```

`filter()`

Filters elements based on condition.

```
numbers = [1, 2, 3, 4, 5]
result = list(filter(lambda x: x % 2 == 0, numbers))
```

`sorted()`

Custom sorting logic.

```
data = [(1, 3), (4, 1), (2, 2)]
result = sorted(data, key=lambda x: x[1])
```

When to Use Lambda Functions

- Short, one-time use functions
- Functional programming
- Cleaner code inside `map()`, `filter()`, `sorted()`

Limitations of Lambda Functions

- Only **one expression (execute one task only)**
- No statements (`if`, `for`, `while`)
- Reduced readability for complex logic

DhruvPatel16120

Recursion in Python

Recursion is a programming technique where a function calls itself either **directly** or **indirectly** to **solve a problem by breaking it into smaller, simpler subproblems**.

In Python, recursion is especially useful for problems that can be divided into **identical smaller tasks**, such as mathematical calculations, tree traversals or divide-and-conquer algorithms.

Working of Recursion : A **recursive function** is just like any other Python function except that **it calls itself in its body**. Let's see basic structure of recursive function:

```
def recursive_function(parameters):
  if base_case_condition:
    return base_result
  else:
    return recursive_function(modified_parameters)
```

Types of Recursion in Python

Recursion can be broadly classified into two types: **tail recursion** and **non-tail recursion**. The main difference between them is related to what happens after recursive call.

- **Tail Recursion:** The recursive call is the last thing the function does, so nothing happens after it returns. Some languages can optimize this to work like a loop, saving memory.
- **Non-Tail Recursion:** The function does more work after the recursive call returns, so it can't be optimized into a loop.

This code compares tail recursion and non-tail recursion using two versions of factorial function one with an accumulator (tail-recursive) and one with multiplication after recursive call (non-tail-recursive).

Two Key Components of Recursion

1. **Base Condition** → stops recursion
2. **Recursive Call** → function calls itself

Without a base condition, recursion leads to **infinite calls**.

Example: Factorial Using Recursion

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

Explanation:

- Base condition: `n == 0`

DhruvPatel16120

- Recursive call: `factorial(n - 1)`

Recursion Flow (Concept)

```
factorial(3)
→ 3 * factorial(2)
→ 2 * factorial(1)
→ 1 * factorial(0)
→ 1
```

Example: Fibonacci Series

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

Recursion vs Loop

Feature	Recursion	Loop
Code readability	High	Medium
Performance	Slower	Faster
Memory usage	More (stack)	Less
Best for	Divide problems	Iteration

Recursion: Recursion is a process in which a function calls itself until a base condition is met.

Common Mistakes in Recursion

- Missing base condition
- Stack overflow
- Excessive recursive calls
- Inefficient for large inputs

When to Use Recursion

- Tree traversal
- Divide-and-conquer algorithms
- Mathematical problems
- Backtracking

Strings in Python

What is a String?

Definition : A string is a sequence of characters used to store and manipulate text. string is an immutable sequence of characters used to store text data in Python.
In Python, strings are immutable, meaning once created, their contents cannot be changed.

Strings are represented using:

DhruvPatel16120

- Single quotes `' '`
- Double quotes `" "`
- Triple quotes `''' '''` or `""" """` (for multi-line strings)

Creating Strings

```
s1 = 'Python'
s2 = "Programming"
s3 = """This is
a multi-line
string"""
```

String Immutability (Core Concept)

Once a string is created, it **cannot be modified**.

Example:

```
text = "Hello"
text[0] = "h"   # Error
```

Correct way:

```
text = "h" + text[1:]
```

String Indexing

Each character in a string has a **position (index)**.

- Positive indexing starts from `0`
- Negative indexing starts from `-1`

Example:

```
word = "Python"
```

Character	P	y	t	h	o	n
Index	0	1	2	3	4	5
Negative Index	-6	-5	-4	-3	-2	-1

Accessing characters:

```
print(word[0])    # P
print(word[-1])   # n
```

String Slicing

Slicing extracts a **portion of a string**.

DhruvPatel16120

Syntax:

```
string[start : end : step]
```

Example:

```
text = "Programming"
print(text[0:6])      # Progra
print(text[::2])      # Pormig
print(text[::-1])     # gnimmargorP logic for reverse string
```

String Length

Use `len()` to get the number of characters.

```
name = "Python"
print(len(name))    # 6
```

String Concatenation and Repetition

Concatenation (+)

```
a = "Hello"
b = "World"
print(a + " " + b)
```

Repetition (*)

```
print("Hi " * 3)
```

Membership in Strings

```
text = "Python Programming"
print("Python" in text)
print("Java" not in text)
```

String Comparison

```
print("apple" == "apple")    # True
print("Apple" > "apple")     # False (compare ASCII based values)
```

String Methods

1. Case Conversion Methods

Method	Description	Example	Output
`upper()`	Converts to uppercase	`"python".upper()`	`PYTHON`
`lower()`	Converts to lowercase	`"PYTHON".lower()`	`python`
`title()`	Capitalizes each word	`"python prog".title()`	`Python Prog`
`capitalize()`	Capitalizes first character	`"python".capitalize()`	`Python`
`swapcase()`	Swaps case	`"PyThOn".swapcase()`	`pYtHoN`

---

2. Whitespace Removal Methods

Method	Description	Example	Output

DhruvPatel16120

strip()	Removes both sides spaces	`" hi ".strip()`	`hi`
lstrip()	Removes left spaces	`" hi".lstrip()`	`hi`
rstrip()	Removes right spaces	`"hi ".rstrip()`	`hi`

---

3. Search and Find Methods

Method	Description	Example	Output
find()	Finds substring index	`"python".find("t")`	`2`
index()	Like find (error if not found)	`"python".index("p")`	`0`
count()	Counts occurrences	`"hello".count("l")`	`2`

---

4. Replace & Modify Methods

Method	Description	Example	Output
replace()	Replaces substring	`"I like Java".replace("Java","Python")`	`I like Python`

---

5. Split and Join Methods

Method	Description	Example	Output
split()	Splits into list	`"a b c".split()`	`['a','b','c']`
rsplit()	Splits from right	`"a b c".rsplit(" ",1)`	`['a b','c']`
join()	Joins list to string	`" ".join(['a','b'])`	`a b`

---

6. String Validation Methods

Method	Checks	Example	Result
isalpha()	Alphabets only	`"Python".isalpha()`	`True`
isdigit()	Digits only	`"123".isdigit()`	`True`
isalnum()	Alphanumeric	`"Py123".isalnum()`	`True`
isspace()	Whitespaces only	`" ".isspace()`	`True`
islower()	Lowercase	`"python".islower()`	`True`
isupper()	Uppercase	`"PY".isupper()`	`True`
startswith()	Starts with	`"file.py".startswith("file")`	`True`
endswith()	Ends with	`"file.py".endswith(".py")`	`True`

7. Alignment and Padding Methods

Method	Description	Example	Output
center()	Centers string	`"hi".center(6,"*")`	`**hi**`
ljust()	Left align	`"hi".ljust(5,"-")`	`hi---`
rjust()	Right align	`"hi".rjust(5,"-")`	`---hi`
zfill()	Pads with zeros	`"42".zfill(5)`	`00042`

---

8. Encoding & Formatting Methods

Method	Description	Example	Output
encode()	Converts to bytes	`"Python".encode()`	`b'Python'`
format()	Formats string	`"Hi {}".format("Alice")`	`Hi Alice`

Dhruvpatel6120

9. Translation Methods

Method	Description	Example	Output
`maketrans()`	Mapping table	`str.maketrans("ae","12")`	mapping
`translate()`	Replace chars	`"apple".translate(t)`	`1ppl2`

Lists in Python

Definition : A list is an **ordered, mutable collection** used to store **multiple values** in a single variable.
Lists can store **different data types** and allow **duplicate values**.

Lists are created using **square brackets `[]`**.

```
numbers = [10, 20, 30]
names = ["Alice", "Bob", "Charlie"]
```

List Characteristics

- Ordered (index-based)
- Mutable (can be modified)
- Allows duplicates
- Can store mixed data types
- Supports nesting

List Indexing (Like String Indexing)

Each element in a list has an **index position**, similar to strings.

Example:

```
lst = [10, 20, 30, 40, 50]
```

Indexing Table

Element	10	20	30	40	50
Index	0	1	2	3	4
Negative Index	-5	-4	-3	-2	-1

Accessing elements:

```
print(lst[0])     # 10
print(lst[-1])    # 50
```

List Slicing (Same as Strings)

Syntax:

```
list[start : end : step]
```

Example:

```
print(lst[1:4])     # [20, 30, 40]
print(lst[::-1])    # [50, 40, 30, 20, 10]
```

DhruvPatel16120

List Mutability (Key Difference from Strings)

Lists can be modified after creation.

```
lst[0] = 100
print(lst)
```

Nested Lists : A nested list is a **list inside another list**.

It is used to represent **matrix-like or hierarchical data**.

Example:

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

Accessing Nested List Elements

```
print(matrix[0])      # [1, 2, 3]
print(matrix[1][2])   # 6
```

Explanation:

- First index selects the row
- Second index selects the column

Nested List Indexing Table

For `matrix = [[1,2,3],[4,5,6]]`

Expression	Output
`matrix[0]`	`[1, 2, 3]`
`matrix[1]`	`[4, 5, 6]`
`matrix[0][1]`	2
`matrix[1][2]`	6

Iterating Over Lists

```
for item in lst:
    print(item)
```

List Comprehension

Definition: List comprehension provides a **short and readable way to create lists** using a single line of code.

Syntax

```
[expression for item in iterable if condition]
```

Simple Example

```
squares = [x * x for x in range(1, 6)]
```

DhruvPatel16120

List Comprehension with Condition

```
even_numbers = [x for x in range(10) if x % 2 == 0]
```

List Comprehension with If–Else

```
result = ["Even" if x % 2 == 0 else "Odd" for x in range(5)]
```

List Comprehension vs Loop

Feature	Loop	List Comprehension
Code length	Longer	Short
Readability	Moderate	High
Performance	Slower	Faster
Pythonic	No	Yes

What Are List Methods?

List methods are **built-in functions** used to **add, remove, modify, search, and organize elements** in a list.

⚠ Important difference from strings:
👉 **Lists are mutable**, so list methods **modify the original list** (most of them).

Categories of List Methods

List methods can be grouped into:

1. Adding elements
2. Removing elements
3. Searching elements
4. Sorting & reversing
5. Copying lists

---

1. Adding Elements to a List

`append()`

Adds **one element at the end** of the list.

```
lst = [1, 2, 3]
lst.append(4)
```

Result: `[1, 2, 3, 4]`

`extend()`

Adds **multiple elements** from another iterable.

```
lst.extend([5, 6])
```

Result: `[1, 2, 3, 4, 5, 6]`

`insert()`

Inserts an element at a **specific index**.

```
lst.insert(1, 10)
```

Result: `[1, 10, 2, 3, 4, 5, 6]`

DhruvPatel1612

2. Removing Elements from a List

`remove()`

Removes the **first occurrence** of a value.

```
lst.remove(10)
```

⚠ Error if value not found.

`pop()`

Removes element by **index** and returns it.

```
lst.pop()
lst.pop(2)
```

`clear()`

Removes **all elements** from the list.

```
lst.clear()
```

Result: `[]`

---

3. Searching and Counting Methods

`index()`

Returns index of first occurrence.

```
lst = [1, 2, 3, 2]
lst.index(2)
```

Output: 1

`count()`

Counts number of occurrences.

```
lst.count(2)
```

Output: 2

4. Sorting and Reversing Methods

`sort()`

Sorts list in **ascending order**.

```
lst = [3, 1, 4, 2]
lst.sort()
```

Sorting in Descending Order

```
lst.sort(reverse=True)
```

`reverse()`

Reverses list order.

```
lst.reverse()
```

---

5. Copying Lists

`copy()`

Creates a **shallow copy** of the list.

```
new_lst = lst.copy()
```

DhruvPatel16120

Using `list()`

```
new_lst = list(lst)
```

⚠ Important:

```
b = a   # not a copy (same reference)
```

Complete List Methods Table

Method	Purpose
`append()`	Add element at end
`extend()`	Add multiple elements
`insert()`	Insert at index
`remove()`	Remove by value
`pop()`	Remove by index
`clear()`	Remove all elements
`index()`	Find index
`count()`	Count occurrences
`sort()`	Sort list
`reverse()`	Reverse list
`copy()`	Copy list

Tuples in Python

Definition : A tuple is an **ordered, immutable collection** used to store multiple values in a single variable.

Once a tuple is created, **its elements cannot be changed**.

Tuples are written using **parentheses `()`**.

```
numbers = (10, 20, 30)
names = ("Alice", "Bob", "Charlie")
```

Tuple Characteristics

- Ordered
- **Immutable**
- Allows duplicate values
- Can store different data types
- Faster than lists (read-only)

Creating Tuples

Normal Tuple

```
t = (1, 2, 3)
```

Single-Element Tuple (Important)

```
t = (10,)   # comma is mandatory
```

Without comma: `t = (10)` # NOT a tuple but int

DhruvPatel

Tuple Indexing (Like String and List)

Each element has an index.

Example:

```
t = (10, 20, 30, 40, 50)
```

Indexing Table

Element	10	20	30	40	50
Index	0	1	2	3	4
Negative Index	-5	-4	-3	-2	-1

Accessing elements:

```
print(t[0])     # 10
print(t[-1])    # 50
```

Tuple Slicing

Tuples support slicing like strings and lists.

```
print(t[1:4])     # (20, 30, 40)
print(t[::-1])    # (50, 40, 30, 20, 10)
```

Tuple Immutability (Key Concept)

Tuple elements **cannot be modified**.

```
t[0] = 100    # Error
```

However, if a tuple contains a mutable object:

```
t = (1, [2, 3])
t[1][0] = 10    # Allowed
```

Edit tuple

Method 1: Tuple concatenation (most common)

```
t = (1, 2, 3)
t = t + (4,)
print(t)
Output:(1, 2, 3, 4)
```

Important detail

```
(4,) is a single-element tuple. The comma is required. Without it:
(4)   # This is just an integer, not a tuple
```

Method 2: Using unpacking (*) — cleaner style

```
t = (1, 2, 3)
t = (*t, 4)
print(t)
This unpacks all elements of t and adds 4 at the end.
You can also add at the beginning: t = (0, *t)
```

Method 3: Convert to list and back (for multiple edits)

```
t = (1, 2, 3)
temp = list(t)
temp.append(4)
t = tuple(temp)
Useful when you need many modifications.
```

Why this works

DhruvPatel16120

Tuples cannot change in memory. Every time you "add" an element, Python actually:

- Creates a **new tuple**
- Copies old elements
- Adds the new element

Quick comparison

Method	Best for	Speed	Readability
Concatenation t + (x,)	Single addition	Fast	Clear
Unpacking (*t, x)	Modern Python style	Fast	Very clean
List conversion	Many edits	Slower	Flexible

Nested Tuples : A tuple inside another tuple is called a **nested tuple**.

```
nested = (1, (2, 3), (4, 5))
```

Accessing nested elements:

```
print(nested[1])      # (2, 3)
print(nested[1][0])   # 2
```

Iterating Over a Tuple

```
for item in t:
    print(item)
```

Tuple Packing and Unpacking

Packing

```
point = (10, 20,30)
```

Unpacking

```
a, b, c = point
```

Unpacking with *

```
a, *b, c = (1, 2, 3, 4, 5)
```

Result:

```
a = 1
b = [2, 3, 4]
c = 5
```

Tuple Methods : Tuples have **only two methods**.

`count()` : Counts occurrences.

```
t = (1, 2, 2, 3)
t.count(2)
```

`index()` : Returns index of first occurrence.

```
t.index(2)
```

DhruvPatel16120

Sets in Python

What is a Set?

Definition : A set is an **unordered, mutable collection of unique elements**.

Sets are used when **duplicate values are not allowed** and fast membership testing is required.

Sets are created using **curly braces `{}`** or the `set()` function.

```
numbers = {1, 2, 3, 4}
```

Key Characteristics of Sets

- **Unordered** (no indexing)
- **No duplicate elements**
- Mutable (can add or remove elements)
- Can store different data types
- Faster lookup compared to lists and tuples

Creating Sets

Using Curly Braces

```
s = {10, 20, 30}
```

Using `set()` Function

```
s = set([1, 2, 3, 3])
```

Result:

```
{1, 2, 3}
```

Empty Set (Important)

```
s = set()      # correct
```

Wrong: `s = {}` # creates an empty dictionary

Set Uniqueness

Sets automatically remove duplicate values.

```
s = {1, 2, 2, 3, 3}
print(s)
```

Output: `{1, 2, 3}`

No Indexing in Sets

Sets do **not support indexing or slicing**.

```
s[0]    # Error
```

Reason: Sets are unordered.

Iterating Over a Set

```
for item in s:
    print(item)
```

Order may vary.

DhruvPatel16120

Set Methods

Adding Elements

`add()`: Adds a single element.

```
s.add(5)
```

`update()` : Adds multiple elements.

```
s.update([6, 7, 8])
```

Removing Elements

`remove()` Removes an element (error if not found).

```
s.remove(3)
```

`discard()` Removes element (no error if not found).

```
s.discard(10)
```

`pop()` Removes a random element.

```
s.pop()
```

`clear()` Removes all elements.

```
s.clear()
```

What Are Set Operations?

Definition: Set operations are used to **compare and combine two or more sets** based on mathematical set theory.

Python supports all major mathematical set operations.

---

1. Union (`|` or `union()`)

Meaning :Returns a set containing **all unique elements from both sets**.

Example

```
A = {1, 2, 3}
B = {3, 4, 5}

print(A | B)
print(A.union(B))
```

Output: `{1, 2, 3, 4, 5}`

---

2. Intersection (`&` or `intersection()`)

Meaning : Returns a set containing **only common elements**.

Example

```
print(A & B)
```

DhruvPatel16120

```
print(A.intersection(B))
```

Output: {3}

---

3. Difference (- or `difference()`)

Meaning : Returns elements present in the **first set but not in the second**.

Example

```
print(A - B)
print(A.difference(B))
```

Output: {1, 2}

---

4. Symmetric Difference (^ or `symmetric_difference()`)

Meaning : Returns elements that are **in either set but not in both**. (! intersection)

Example

```
print(A ^ B)
print(A.symmetric_difference(B))
```

Output: {1, 2, 4, 5}

---

5. Subset (`issubset()`)

Meaning : Checks whether **all elements of one set exist in another set**.

Example

```
C = {1, 2}
print(C.issubset(A))
```

Output: True

---

6. Superset (`issuperset()`)

Meaning : Checks whether a set contains **all elements of another set**.

Example

```
print(A.issuperset(C))
```

Output: True

---

7. Disjoint Sets (`isdisjoint()`)

Meaning : Checks whether two sets have **no common elements**.

Example

```
D = {7, 8}
```

DhruvPatel16120

```
print(A.isdisjoint(D))
```
Output: True

Operator vs Method Summary Table

Operation	Operator	Method
Union	`	`
Intersection	&	`intersection()`
Difference	-	`difference()`
Symmetric Difference	^	`symmetric_difference()`
Subset	—	`issubset()`
Superset	—	`issuperset()`
Disjoint	—	`isdisjoint()`

Frozen Set

Frozen set : a `frozenset` is an immutable version of a set.

```
fs = frozenset([1, 2, 3])
```

Use case:

- Dictionary keys
- Fixed set data

Dictionary in Python

Definition : A dictionary is a mutable, unordered collection of data that stores values in key–value pairs.

- Each **key** is unique
- Each key maps to a **value**
- Dictionaries are used to represent **structured data**

Dictionaries are written using **curly braces** `{}` with `key: value` pairs.

```
student = {
    "name": "Alice",
    "age": 20,
    "grade": "A"
}
```

Key Characteristics of Dictionary

- Stores data as **key–value pairs**
- **Mutable** (can be changed)
- **Keys must be unique and immutable**
- Values can be of any data type
- Fast lookup using keys

Creating Dictionaries

Using Curly Braces

```
person = {"name": "Bob", "age": 25}
```

Using `dict()` Function

```
person = dict(name="Bob", age=25)
```

DhruvPatel16120

Empty Dictionary

```
d = {}
```

Dictionary Keys and Values

Valid Keys

- int
- float
- string
- tuple

✕ Invalid keys:

- list
- set
- dictionary

Example:

```
d = {(1, 2): "tuple key"}   # valid
```

Accessing Dictionary Elements

Using Key

```
print(student["name"])
```

⚠ Error if key does not exist.

Using `get()` (Safe Method)

```
print(student.get("age"))
print(student.get("marks", "Not Found"))
```

Modifying Dictionary

Adding New Key–Value Pair

```
student["city"] = "Delhi"
```

Updating Existing Value

```
student["age"] = 21
```

Removing Dictionary Elements

`pop()` : Removes item by key.

```
student.pop("grade")
```

DhruvPatel16120

`popitem()` : Removes **last inserted** key–value pair.

```
student.popitem()
```

`del` : Deletes a key or entire dictionary.

```
del student["age"]
```

`clear()` : Removes all elements.

```
student.clear()
```

Iterating Through Dictionary

Loop Through Keys

```
for key in student:
    print(key)
```

Loop Through Values

```
for value in student.values():
    print(value)
```

Loop Through Key–Value Pairs

```
for key, value in student.items():
    print(key, value)
```

Dictionary Methods :

`keys()` : Returns all keys.

```
student.keys()
```

`values()` : Returns all values.

```
student.values()
```

`items()` : Returns key–value pairs.

```
student.items()
```

`update()` : Updates dictionary with another dictionary.

```
student.update({"marks": 90})
```

`copy()` : Creates a shallow copy.

```
new_student = student.copy()
```

`setdefault()` : Returns value if key exists, else inserts key.

```
student.setdefault("country", "India")
```

DhruvPatel16120

Nested Dictionaries

A dictionary inside another dictionary.

```
students = {
    1: {"name": "Alice", "age": 20},
    2: {"name": "Bob", "age": 22}
}
```

Accessing nested data:

```
print(students[1]["name"])
```

Dictionary Comprehension

Definition : Dictionary comprehension provides a **short and readable way** to create dictionaries.

Syntax : `{key: value for item in iterable if condition}`

Example : `squares = {x: x*x for x in range(1, 6)}`

Dictionary vs Other Collections

Feature	Dictionary	List	Tuple	Set
Structure	Key–Value	Values	Values	Values
Mutable	Yes	Yes	No	Yes
Indexing	Keys	Index	Index	No
Duplicates	Keys ✕	Yes	Yes	No

Common Mistakes

- Using mutable types as keys
- Accessing missing keys without `get()`
- Confusing `{}` as empty set
- Assuming dictionary is ordered (older Python versions)

DhruvPatel5120